PROGRAM 1

Linear Search – Function Returning Value

1.1 Aim

To implement Linear Search using a function that returns the position (index) of the key element in the array, and to analyze its time and space complexity.

1.2 Theory

What is Searching?

Searching is the process of finding a particular element (called the key or target) within a collection of data. It is one of the most fundamental operations in computer science. The result of a search is either the position of the element in the collection or a report that the element is not present.

Linear Search

Linear Search (also called Sequential Search) is the simplest searching algorithm. It works by examining each element in the array one by one, from the first to the last, until the desired key is found or the entire array has been scanned.

Linear Search does NOT require the array to be sorted. It works on both sorted and unsorted arrays, which is its main advantage over Binary Search.

How Linear Search Works

Start from the first element (index 0).
Compare the current element with the key.
If they match → return the position (found).
If they do not match → move to the next element.
Repeat until the key is found or the array is exhausted.
If the key is not found after scanning all elements → return -1 (not found).

Functions in C/C++

A function is a reusable block of code that performs a specific task. Functions help in modular programming — breaking a large program into smaller, manageable units.

Concept	Explanation
Function Declaration (Prototype)	Tells the compiler about the function name, return type, and parameters before it is defined.
Function Definition	The actual body of the function containing the code to be executed.
Function Call	The statement that invokes/executes the function from main() or another function.
Formal Parameters	Variables listed in the function definition that receive values when the function is called.
Actual Parameters	Values or variables passed to the function at the time of the call.
Return Value	The value a function sends back to the calling function using the return statement.

Arrays and Functions

When an array is passed to a function in C/C++, the address of the first element is passed (pass by reference). This means the function can access and modify the original array. The size of the array must also be passed separately since the function has no way to determine the array size on its own.

Asymptotic Notations

Asymptotic notation is used to describe the performance (efficiency) of an algorithm as the input size grows. The three most common notations are:

Notation	Meaning
Big-O O(f(n))	Upper bound — describes the worst-case time complexity.
Big-Omega Ω(f(n))	Lower bound — describes the best-case time complexity.
Big-Theta Θ(f(n))	Tight bound — describes both upper and lower bound (average case).

1.3 Algorithm

FillArray(a, n)

```
Algorithm FillArray(a, n)
Step 1: For i ← 0 to n-1 do
        Read a[i]
    End for
Step 2: Exit
```

DisplayArray(a, n)

```
Algorithm DisplayArray(a, n)
Step 1: For i ← 0 to n-1 do
        Print a[i]
    End for
Step 2: Exit
```

Linear Search — LS(a, n, key)

```
Algorithm LS(a, n, key)
// a is an array of size n, key is the element to search
Step 1: Initialize i ← 0
Step 2: Repeat while i < n
        If a[i] = key
           Then return i + 1   // 1-based position
        End if
        i ← i + 1
    End while
Step 3: Return -1   // key not found
```

Main Algorithm

```
Algorithm Main()
Step 1: Read n
Step 2: Declare array a of size n
Step 3: Call FillArray(a, n)
Step 4: Call DisplayArray(a, n)
Step 5: Read key
Step 6: r ← LS(a, n, key)
Step 7: If r = -1
        Print 'Key not found'
    Else
        Print 'Key found at position', r
    End if
Step 8: Exit
```

1.4 Sample Output

Test Case 1 — Key Found

```
Array Size: 5
Enter Array Elements: 6 1 4 3 2
The Array Elements are: 6 1 4 3 2
Enter key to search: 4
Key found at pos 3
```

Test Case 2 — Key Not Found

```
Array Size: 5
Enter Array Elements: 6 2 4 3 1
The Array Elements are: 6 2 4 3 1
Enter key to search: 8
Key not found
```

1.5 Time & Space Complexity

Case	Time Complexity	Space Complexity
Best Case	O(1) — Key is the first element	O(n) — Array storage
Worst Case	O(n) — Key is last or absent	O(n) — Array storage
Average Case	O(n)	O(n) — Array storage

Note: The auxiliary space complexity (extra space used by variables only) is O(1).
Only a fixed number of variables are used: i, key, n, r — regardless of input size.
The O(n) space is for the input array itself, which is unavoidable.

1.6 MCQs

#	Question	Options	Answer
1	When an array is passed to a function, what happens to its elements?	A) The original array is accessed and can be modified B) Only the first element is passed C) A new copy is created D) It becomes constant	**A**
2	What is required to correctly process an array inside a function?	A) Only array name B) Only size of array C) Both array and its size D) Neither	**C**
3	What are formal parameters?	A) Variables defined in the function declaration/definition B) Values passed to a function C) Output of a function D) Constants	**B**
4	What does Big-O notation represent?	A) Best-case performance B) Worst-case performance C) Average performance D) Memory allocation	**B**
5	Which algorithm requires the array to be sorted?	A) Linear Search B) Traversal C) Selection Sort D) Binary Search	**D**
6	Time complexity of accessing an element by index?	A) O(n) B) O(log n) C) O(1) D) $O(n^2)$	**C**
7	Which is NOT a characteristic of a good algorithm?	A) Finiteness B) Ambiguity C) Effectiveness D) Input/Output	**B**
8	If the number of operations is constant, the complexity is:	A) O(n) B) O(log n) C) O(1) D) $O(n^2)$	**C**
9	What will happen if a function does not return any value?	A) It causes an error B) It must have return type void C) It automatically returns 0 D) It stops execution	**B**
10	Which correctly declares a function?	A) int function(); B) function int(); C) int function; D) function();	**A**

1.7 Short Questions & Answers

#	Question	Answer
1	**What is Linear Search?**	Linear Search is a sequential searching algorithm that checks each element of the array one by one until the key is found or the array ends.
2	**What is the difference between formal and actual parameters?**	Formal parameters are variables in the function definition. Actual parameters are the values passed when the function is called.
3	**Why must array size be passed separately to a function?**	When an array is passed, only its base address is passed. The function cannot determine the number of elements from the address alone, so the size must be passed explicitly.
4	**What does a function with return type int return?**	It returns an integer value back to the calling function using the return statement.
5	**When does Linear Search give O(1) best-case complexity?**	When the key element is located at the very first position (index 0) of the array.

1.8 Numericals

#	Problem	Solution
1	**An array has 10 elements. In the worst case, how many comparisons does Linear Search make? In the best case?**	Worst Case: The key is the last element or not present. Number of comparisons = n = 10 Best Case: The key is the first element. Number of comparisons = 1 Answer: Worst = 10, Best = 1
2	**Given array: [5, 3, 8, 1, 9, 2]. How many comparisons are needed to find key = 9?**	Compare $5 \neq 9 \rightarrow 1$ Compare $3 \neq 9 \rightarrow 2$ Compare $8 \neq 9 \rightarrow 3$ Compare $1 \neq 9 \rightarrow 4$ Compare $9 = 9 \rightarrow 5$ (found at position 5) Answer: 5 comparisons
3	**If an algorithm's time complexity is O(n), how many operations are needed for n = 1000?**	For O(n): operations $\propto$ n For n = 1000: approximately 1000 operations Answer: ~1000 operations

```cpp
/*AIM- To implement Linear Search using a function that returns the p
to analyze its time and space complexity.*/
//PROGRAMMER NAME = POONIS

#include<iostream>
using namespace std;

void fillArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << "Enter " << i << " Element : ";
        cin >> arr[i];
    }
}

void displayArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << arr[i] << " ";
    }
}

int linearSearch(int arr[], int size, int key){
    for(int i = 0; i < size; i++){
        if(arr[i] == key){
            return i;
        }
    }
    return -1;
}

int main(){
    int size, key;

    cout << "Enter Size of Array : ";
    cin >> size;

    int arr[size];

    fillArray(arr, size);

    cout << "Array Elements : ";
    displayArray(arr, size);
    cout << endl;

    cout << "Enter Key Element : ";
    cin >> key;

    int result = linearSearch(arr, size, key);

    if(result != -1){
        cout << "Key Found at pos : " << result+1;
    }
    else{
        cout << "Key Not Found";
    }

    return 0;
}
```

```
PS C:\out> C:\out\linearSearch.exe
Enter 3 Element : 3
Enter 4 Element : 2
Array Elements : 6 1 4 3 2
Enter Key Element : 4
Key Found at pos : 3
PS C:\out> C:\out\linearSearch.exe
Enter Size of Array : 5
Enter 0 Element : 6
Enter 1 Element : 2
Enter 2 Element : 4
Enter 3 Element : 3
Enter 4 Element : 1
Array Elements : 6 2 4 3 1
Enter Key Element : 8
Key Not Found
```

PROGRAM 2

Linear Search – Void Function (No Return Value)

2.1 Aim

To implement Linear Search using a void function that prints the result directly (instead of returning a value), and to understand the difference between void and non-void functions.

2.2 Theory

Void Functions vs Value-Returning Functions

In C/C++, functions are broadly classified based on their return type:

Type	Behavior
Value-Returning Function (e.g., int, float)	Performs computation and returns a result to the caller using the return statement. The caller can store or use this returned value.
Void Function	Performs a task (like printing output) but does NOT return any value. The return type is declared as void. The function call appears as a standalone statement.

In Program 1, the LS() function returned the position as an integer. In this program, the LS() function itself prints whether the key was found or not, and the main function simply calls it without storing any return value.

Why Use Void Functions?

When the function performs an action (like printing) rather than computing a value.
When the result is communicated through output statements or modified parameters.
Simplifies the calling code — no need to handle a return value.
Useful for procedures such as display, initialization, or sorting.

Pass by Value vs Pass by Reference

Pass By Value	Pass By Reference
A copy of the variable is passed.	The address (reference) of the variable is passed.
Changes inside function do NOT affect original.	Changes inside function DO affect the original.
Used for simple variables (int, float, etc.).	Used for arrays and when output is needed.
Syntax: void fun(int x)	Syntax: void fun(int &x) or void fun(int a[])

2.3 Algorithm

FillArray(a, n)	DisplayArray(a, n)
Algorithm FillArray(a, n) Step 1: For i ← 0 to n-1 do Read a[i] End for Step 2: Exit	Algorithm DisplayArray(a, n) Step 1: For i ← 0 to n-1 do Print a[i] End for Step 2: Exit

Linear Search — LS(a, n, key)	**Main Algorithm**

Algorithm LS(a, n, key) // void function
Step 1: Set i ← 0
Step 2: Repeat while i < n
If a[i] = key
Then
Print 'Key found at position', i + 1
Go to Step 4
End if
i ← i + 1
End while
Step 3: If i = n
Print 'Key not found'
Step 4: Exit
	Algorithm Main()
	Step 1: Read n
	Step 2: Declare array a of size n
	Step 3: Print 'Enter Array Elements'
	Call FillArray(a, n)
	Step 4: Print 'The Array Elements are'
	Call DisplayArray(a, n)
	Step 5: Read key
	Step 6: Call LS(a, n, key) // no return value captured
	Step 7: Exit

2.4 Sample Output

Test Case 1 — Key Found

```
Array Size: 7
Enter Array Elements: 7 6 5 4 3 2 1
The Array Elements are: 7 6 5 4 3 2 1
Enter key to search: 3
Key found at pos 5
```

Test Case 2 — Key Not Found

```
Array Size: 4
Enter Array Elements: 6 3 2 9
The Array Elements are: 6 3 2 9
Enter key to search: 12
Key not found
```

2.5 Time & Space Complexity

Case	**Time Complexity**	**Space Complexity**
Best Case	O(1) — Key at index 0	O(1) auxiliary space
Worst Case	O(n) — Key not present	O(1) auxiliary space
Average Case	O(n)	O(1) auxiliary space

2.6 MCQs

#	Question	Options	Answer
1	What is the time complexity of Linear Search in worst case?	A) O(1) B) O(log n) C) O(n^2) D) O(n)	**D**
2	In Linear Search, the best case occurs when:	A) Element is not present B) Element is at the last position C) Element is at the first position D) Array is sorted	**C**
3	Which notation represents worst-case complexity?	A) Big Omega Ω B) Big Theta Θ C) Big O (O) D) Sigma Σ	**C**
4	How is an array passed to a function in C/C++?	A) By reference (address) B) By value C) By copying all elements D) By pointer arithmetic only	**A**
5	What does int a[] represent in a function parameter?	A) A single integer B) A pointer to the first element C) A constant value D) A loop variable	**B**
6	What is a void function?	A) A function that returns 0 B) A function that cannot be called C) A function with no parameters D) A function that returns no value	**D**
7	What is the space complexity of the array a[n]?	A) O(1) B) O(log n) C) O(n) D) O(n^2)	**C**
8	What is a function?	A) A variable declaration B) A block of code performing a specific task C) A loop structure D) A data type	**B**
9	Correct declaration for passing an array to a function?	A) void fun(int a[]); B) void fun(int a); C) void fun(a[] int); D) void fun(int);	**A**
10	In pass by reference, what is passed?	A) Copy of value B) New variable C) Address of the variable D) Nothing	**C**

2.7 Short Questions & Answers

#	Question	Answer
1	**What is a void function?**	A void function is a function that does not return any value. It performs a task (such as printing output) and the return type is declared as void.
2	**How does Program 2 differ from Program 1?**	In Program 1, LS() returns the position as an integer. In Program 2, LS() is a void function — it prints the result directly and returns nothing.
3	**What is auxiliary space?**	Auxiliary space is the extra memory used by an algorithm excluding the input data. For Linear Search, auxiliary space = O(1) since only a few extra variables are used.
4	**What is pass by reference?**	In pass by reference, the memory address of a variable is passed to a function. The function can then directly access and modify the original variable.
5	**Why is the space complexity O(1) for the void linear search?**	The void linear search only uses a constant number of extra variables (i, key, n) regardless of the input size. The input array is not counted as extra space created by the algorithm.

2.8 Numericals

#	Problem	Solution
1	**For an array of size n = 20, what is the average number of comparisons in Linear Search?**	Average Case: (n + 1) / 2 comparisons (when element is present). For n = 20: (20 + 1) / 2 = 21 / 2 = 10.5 ≈ 11 comparisons. Answer: ~11 comparisons on average
2	**Array: [10, 20, 30, 40, 50]. Key = 30. Count comparisons and state time complexity.**	Compare 10 ≠ 30 → 1, Compare 20 ≠ 30 → 2, Compare 30 = 30 → 3 (found at position 3) This is the average case: O(n) with 3 comparisons out of 5. Answer: 3 comparisons, Time = O(n)
3	**If T(n) = 5n + 3, what is the Big-O notation?**	Drop constant terms and lower-order terms. T(n) = 5n + 3 → dominant term = 5n , Drop the constant coefficient 5. Big-O = O(n) Answer: O(n)

```cpp
/*AIM- To implement Linear Search using a void function that prints t
to understand the difference between void and non-void functions.*/
//PROGRAMMER NAME = POONIS

#include<iostream>
using namespace std;

void fillArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << "Enter " << i << " Element : ";
        cin >> arr[i];
    }
}

void displayArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << arr[i] << " ";
    }
}

// Linear Search using Void Function
void linearSearch(int arr[], int size, int key){
    for(int i = 0; i < size; i++){
        if(arr[i] == key){
            cout << "Key Found at Position : " << i + 1;
            return;
        }
    }

    cout << "Key Not Found";
}

int main(){
    int size, key;

    cout << "Enter Size of Array : ";
    cin >> size;

    int arr[size];

    cout << "Enter Array Elements : " << endl;
    fillArray(arr, size);

    cout << "Array Elements are : ";
    displayArray(arr, size);
    cout << endl;

    cout << "Enter Key Element : ";
    cin >> key;

    linearSearch(arr, size, key);

    return 0;
}
```

```
PS C:\out> C:\out\linearSearch.exe
Enter Size of Array : 7
Enter Array Elements :
Enter 0 Element : 7
Enter 1 Element : 6
Enter 2 Element : 5
Enter 3 Element : 4
Enter 4 Element : 3
Enter 5 Element : 2
Enter 6 Element : 1
Array Elements are : 7 6 5 4 3 2 1
Enter Key Element : 3
Key Found at Position : 5
PS C:\out> C:\out\linearSearch.exe
Enter Size of Array : 4
Enter Array Elements :
Enter 0 Element : 6
Enter 1 Element : 3
Enter 2 Element : 2
Enter 3 Element : 9
Array Elements are : 6 3 2 9
Enter Key Element : 12
Key Not Found
```

PROGRAM 3

Iterative Binary Search

3.1 Aim

To implement the Binary Search algorithm using an iterative approach (using loops) and to analyze its time and space complexity.

3.2 Theory

What is Binary Search?

Binary Search is an efficient searching algorithm that works on sorted arrays. It follows the Divide and Conquer strategy: at each step, it divides the search space in half by comparing the key with the middle element.

Prerequisite: Sorted Array

Binary Search REQUIRES the array to be sorted in ascending (or descending) order.
If the array is unsorted, Binary Search will give incorrect or unpredictable results.
Advantage over Linear Search: Binary Search is significantly faster for large arrays — O(log n) vs O(n).

How Binary Search Works (Iterative)

The search space is defined by two pointers: first (left boundary) and last (right boundary).

Step	Action
1. Compute mid	mid = ⌊(first + last) / 2⌋
2. Compare a[mid] with key	If equal → key found at mid+1
3. Key < a[mid]	Search LEFT half → last = mid - 1
4. Key > a[mid]	Search RIGHT half → first = mid + 1
5. Repeat	Until first > last (key not found)

Iterative vs Recursive

Iterative Binary Search	Recursive Binary Search
Uses a while loop	Uses function calls
Space complexity = O(1)	Space complexity = O(log n) due to call stack
No overhead of function calls	Has overhead of function calls
Generally preferred for Binary Search	Elegant and easier to understand

Static vs Dynamic Arrays

Static Array (int a[10];)	Dynamic Array (int a[n];)
Size fixed at compile time	Size determined at runtime based on user input
Memory allocated on the stack	Memory allocated on the stack (VLA) or heap (malloc/new)
Cannot resize	More flexible — size depends on input
Example: int a[10];	Example: int a[n]; where n is read from user

3.3 Algorithm

FillArray(a, n) (sorted Data Only)	DisplayArray(a, n)

Algorithm FillArray(a, n)
 	
Step 1: Read a[0]
 	
Step 2: For i ← 1 to n-1 do Repeat
Read a[i]
If a[i] < a[i-1] then
Print "Enter a value greater than or equal to ", a[i-1]
End if
Until a[i] >= a[i-1]
End for
 	
Step 3: Exit
	Algorithm DisplayArray(a, n)
	Step 1: For i ← 0 to n-1 do
	Print a[i]
	End for
	Step 2: Exit

Binary Search — BS(a, first, last, key)	**Main Algorithm**

Algorithm BS(a, first, last, key)
// a is a sorted array
Step 1: Repeat while first ≤ last
Step 2: mid ← ⌊(first + last) / 2⌋
Step 3: If a[mid] = key
Then return mid + 1 // 1-based position
Step 4: If key < a[mid]
Then last ← mid - 1 // search left subarray
Else
first ← mid + 1 // search right subarray
Step 5: End while
Step 6: Return -1 // key not found
	Algorithm Main()
	Step 1: Read n
	Step 2: Declare array a of size n
	Step 3: Call FillArray(a, n)
	Step 4: Call DisplayArray(a, n)
	Step 5: Read key
	Step 6: r ← BS(a, 0, n-1, key)
	Step 7: If r = -1
	Print 'Key not found'
	Else
	Print 'Key found at position', r
	Step 8: Exit

D3.4 Sample Output

Test Case 1 — Key Found

```
Array Size: 5
Enter Array Elements: 1 2 3 4 5
The Array Elements are: 1 2 3 4 5
Enter key to search: 4
Key found at pos 4
```

Test Case 2 — Key Not Found

```
Array Size: 5
Enter Array Elements: 1 2 3 4 5
The Array Elements are: 1 2 3 4 5
Enter key to search: 12
Key not found
```

3.5 Time & Space Complexity

Case	Time Complexity	Space Complexity
Best Case	O(1) — Key is at the middle element	O(1) — only variables used
Worst Case	O(log n) — Key not found or at extremes	O(1) — only variables used
Average Case	O(log n)	O(1) — only variables used

Why O(log n)? Each comparison halves the search space:
After 1 comparison: n/2 elements remain
After 2 comparisons: n/4 elements remain
After k comparisons: n/2^k elements remain
Search ends when n/2^k = 1 → k = $\log_2(n)$
Therefore, maximum comparisons = $\log_2(n)$ → Time Complexity = O(log n)

3.6 MCQs

#	Question	Options	Answer
1	Binary Search works on:	A) Sorted array B) Unsorted array C) Linked list only D) Stack	**A**
2	Worst-case time complexity of Binary Search is:	A) O(n) B) O(log n) C) $O(n^2)$ D) O(1)	**B**
3	Best-case time complexity of Binary Search is:	A) O(n) B) O(log n) C) O(1) D) $O(n^2)$	**C**
4	Space complexity of iterative Binary Search is:	A) O(n) B) O(log n) C) $O(n^2)$ D) O(1)	**D**
5	Space complexity of recursive Binary Search is:	A) O(1) B) O(n) C) O(log n) D) $O(n^2)$	**C**
6	int a[10]; is an example of:	A) Dynamic array B) Static array C) Pointer D) Function	**B**
7	When key > a[mid], the next search is in:	A) Right half B) Left half C) Entire array D) Middle only	**A**
8	When key < a[mid], the next search is in:	A) Right half B) Entire array C) Left half D) Random position	**C**
9	Function definition means:	A) Calling a function B) Declaring variables C) Writing actual body of function D) Passing arguments	**C**
10	Binary Search performs how many comparisons for n = 1024?	A) 1024 B) 512 C) 10 D) 1	**C**

3.7 Short Questions & Answers

#	Question	Answer
1	**Why must Binary Search array be sorted?**	Binary Search compares the key with the middle element and decides which half to search next. This decision is only valid if the array is sorted — in an unsorted array, half the elements might be discarded incorrectly.
2	**What is the mid formula in Binary Search?**	mid = ⌊(first + last) / 2⌋. The floor function ensures mid is an integer index.
3	**Why is Binary Search O(log n)?**	Each comparison halves the search space. Starting with n elements, after k steps only n/2^k remain. When n/2^k = 1, k = $\log_2 n$. So at most $\log_2 n$ comparisons are made.
4	**What is the difference in space complexity between iterative and recursive Binary Search?**	Iterative Binary Search: O(1) — only uses loop variables. Recursive Binary Search: O(log n) — each recursive call adds a frame to the call stack.
5	**What happens when first > last in Binary Search?**	The condition first > last indicates that the search space is empty. The key is not present in the array, so the algorithm returns -1.

3.8 Numericals

#	Problem	Solution
1	**For a sorted array of 64 elements, what is the maximum number of comparisons Binary Search needs?**	Maximum comparisons = $\log_2(n)$ $\log_2(64) = \log_2(2^6) = 6$ Answer: 6 comparisons maximum
2	**Trace Binary Search on array [1, 3, 5, 7, 9, 11, 13] for key = 7.**	first=0, last=6, mid=3, a[3]=7 = key → FOUND! Position = mid + 1 = 3 + 1 = 4 Answer: Found at position 4 in just 1 comparison (best case!)
3	**Compare Linear Search vs Binary Search for n = 1,000,000. Worst case comparisons?**	Linear Search: O(n) → 1,000,000 comparisons Binary Search: O(log n) → $\log_2(1{,}000{,}000) \approx 20$ comparisons Binary Search is ~50,000 times faster! Answer: Linear = 1,000,000 | Binary = ~20

```cpp
/*AIM- To implement the Binary Search algorithm using an iterative ap
space complexity.*/
//PROGRAMMER NAME = POONIS

#include<iostream>
using namespace std;

void fillArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << "Enter " << i << " Element : ";
        cin >> arr[i];

        if(i > 0 && arr[i] < arr[i - 1]){
            cout << "Data is Unsorted. Enter a value greater than or
                 << arr[i - 1] << endl;
            i--;
        }
    }
}

void displayArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << arr[i] << " ";
    }
}

int binarySearch(int arr[], int first, int last, int key){
    while(first <= last){
        int mid = first + (last - first) / 2;

        if(arr[mid] == key){
            return mid;
        }
        else if(key < arr[mid]){
            last = mid - 1;
        }
        else{
            first = mid + 1;
        }
    }

    return -1;
}

int main(){
    int size, key;

    cout << "Enter Size of Array : ";
    cin >> size;

    int arr[size];

    fillArray(arr, size);

    cout << "Array Elements : ";
    displayArray(arr, size);
    cout << endl;

    cout << "Enter Key Element : ";
    cin >> key;

    int result = binarySearch(arr, 0, size - 1, key);

    if(result != -1){
        cout << "Key Found at pos : " << result+1;
    }
    else{
        cout << "Key Not Found";
    }

    return 0;
}
```

```
PS C:\out> C:\out\BinarySearch.exe
Enter Size of Array : 5
Enter 0 Element : 1
Enter 1 Element : 2
Enter 2 Element : 3
Enter 3 Element : 4
Enter 4 Element : 5
Array Elements : 1 2 3 4 5
Enter Key Element : 4
Key Found at pos : 4
PS C:\out> C:\out\BinarySearch.exe
Enter Size of Array : 5
Enter 0 Element : 1
Enter 1 Element : 2
Enter 2 Element : 3
Enter 3 Element : 4
Enter 4 Element : 5
Array Elements : 1 2 3 4 5
Enter Key Element : 12
Key Not Found
```

PROGRAM 4

Recursive Binary Search

4.1 Aim

To implement the Binary Search algorithm using a recursive approach and to analyze its time and space complexity using recurrence relations.

4.2 Theory

What is Recursion?

Recursion is a programming technique where a function calls itself to solve a smaller version of the same problem. Each recursive call reduces the problem size until a base condition is met, at which point the recursion stops.

Key Components of Recursion

Component	Description
Base Condition	The stopping condition that prevents infinite recursion. Without it, the function would call itself indefinitely causing a stack overflow.
Recursive Case	The part of the function where it calls itself with a smaller/simpler input.
Call Stack	Each recursive call is pushed onto the system call stack. Memory is used for each frame. When the base condition is reached, frames are popped off.
Stack Overflow	Occurs when recursion is too deep (no base case or very large input), exceeding the stack memory limit.

Recursive Binary Search — How It Works

In the recursive version, instead of a loop, the function calls itself with updated boundaries (first and last) to search the appropriate half of the array.

```
BSR(a, first, last, key):
  IF first > last  → return -1        (Base Case: not found)
  Compute mid = (first + last) / 2
  IF a[mid] = key  → return mid + 1    (Base Case: found)
  IF key < a[mid]  → BSR(a, first, mid-1, key)   (Search left)
  ELSE             → BSR(a, mid+1, last, key)    (Search right)
```

Recurrence Relation & Time Complexity

$T(n) = T(n/2) + c$
Where: $T(n/2)$ → one recursive call on half the array, c → constant work per call (compute mid, compare)
Solving by Substitution:
$T(n) = T(n/2) + c$ ➔ $T(n/4) + 2c$ ➔ $T(n/8) + 3c$ ➔ $= T(n/2^k) + kc$
Stop when $n/2^k = 1$ → $k = \log_2 n$
$T(n) = T(1) + c \cdot \log_2 n = c \cdot \log n + c$
$\therefore T(n) = O(\log n)$

4.3 Algorithm

FillArray(a, n) (sorted Data Only)	**DisplayArray(a, n)**

Algorithm FillArray(a, n)
Step 1: Read a[0]
Step 2: For i ← 1 to n-1 do
Repeat
Read a[i]
If a[i] < a[i-1] then
Print "Enter a value greater than or equal to ", a[i-1]
End if
Until a[i] >= a[i-1]
End for
Step 3: Exit
	Algorithm DisplayArray(a, n)
	Step 1: For i ← 0 to n-1 do
	Print a[i]
	End for
	Step 2: Exit

Binary Search — BS(a, first, last, key)	**Main Algorithm**

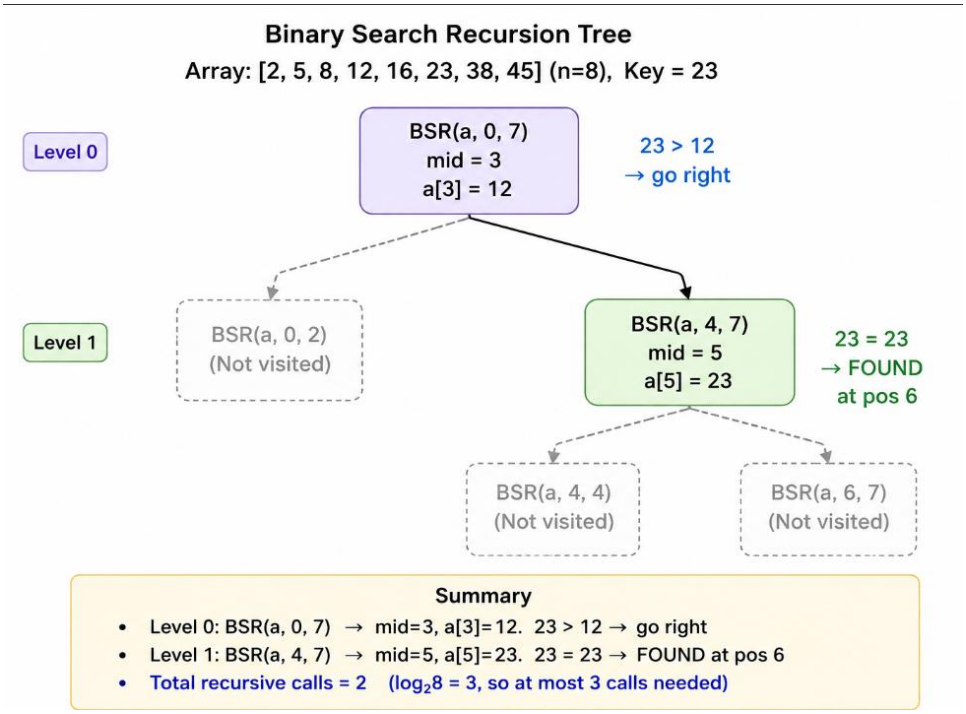
Algorithm BSR(a, first, last, key)
// a is a sorted array
Step 1: If first > last
Then return -1 // Base case: unsuccessful search
Step 2: mid ← ⌊(first + last) / 2⌋
Step 3: If a[mid] = key
Then return mid + 1 // Base case: successful search
Step 4: If key < a[mid]
Then return BSR(a, first, mid - 1, key) // Search left
Step 5: Else
return BSR(a, mid + 1, last, key) // Search right
	Algorithm Main()
	Step 1: Read n
	Step 2: Declare array a of size n
	Step 3: Call FillArray(a, n)
	Step 4: Call DisplayArray(a, n)
	Step 5: Read key
	Step 6: r ← BSR(a, 0, n-1, key)
	Step 7: If r = -1
	Print 'Key not found'
	Else
	Print 'Key found at position', r
	Step 8: Exit

4.4 Sample Output

Test Case 1 — Key Found	Test Case 2 — Key Not Found
Array Size: 5 Enter Array Elements: 1 2 3 4 5 The Array Elements are: 1 2 3 4 5 Enter key to search: 4 Key found at pos 4	Array Size: 5 Enter Array Elements: 1 2 3 4 5 The Array Elements are: 1 2 3 4 5 Enter key to search: 12 Key not found

4.5 Time & Space Complexity

Case	**Time Complexity**	**Space Complexity**
Best Case	O(1) — Key found at first mid	O(log n) — call stack depth
Worst Case	O(log n) — Key not found	O(log n) — call stack depth
Average Case	O(log n)	O(log n) — call stack depth



Why O(log n) space? Unlike iterative Binary Search which uses O(1) space,
the recursive version places a new stack frame for each recursive call.
Since the maximum recursion depth = $\log_2 n$, the space complexity = O(log n).
Comparison: Iterative = O(1) space | Recursive = O(log n) space. Both have the same time complexity: O(log n).

4.6 MCQs

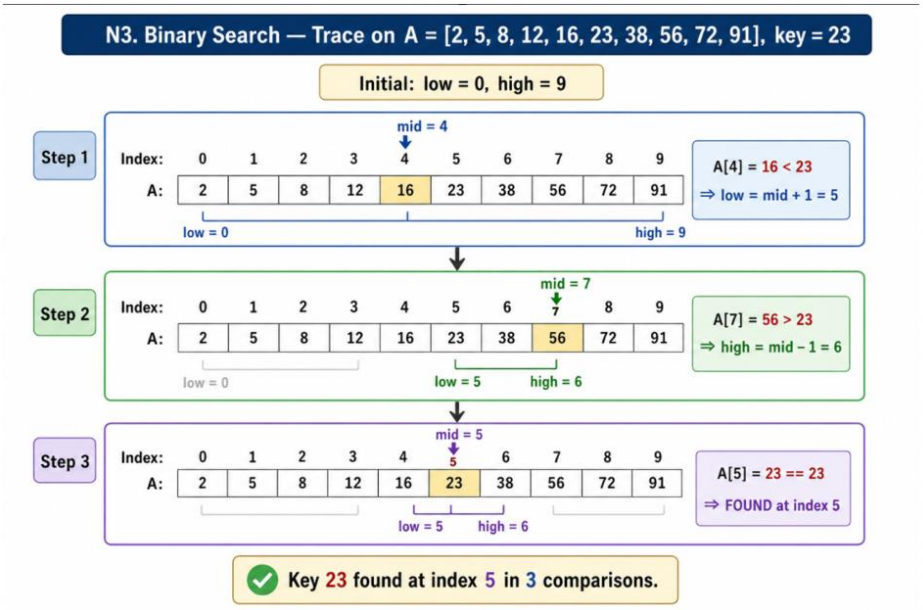
#	Question	Options	Answer
1	Actual parameters are:	A) Values passed during function call B) Variables inside function C) Constants only D) Loop variables	**A**
2	In pass by value:	A) Original value changes B) Copy of value is passed C) Address is passed D) No value is passed	**B**
3	In pass by reference:	A) Copy is passed B) Value cannot change C) Only constants are passed D) Address is passed	**D**
4	Recursion is:	A) Loop inside loop B) Function calling itself C) Function returning value D) Infinite loop	**B**
5	Base condition is necessary to:	A) Increase execution time B) Stop recursion C) Call another function D) Skip execution	**B**
6	If base condition is missing:	A) Program stops B) Program runs faster C) Output is correct D) Infinite recursion occurs	**A**
7	If key < a[mid], we search:	A) Left half B) Right half C) Entire array D) Random	**A**
8	If key > a[mid], we search:	A) Left half B) Entire array C) Middle only D) Right half	**D**
9	Space complexity of recursive Binary Search:	A) O(log n) B) O(n) C) O(1) D) $O(n^2)$	**A**
10	Recursive Binary Search differs from iterative because:	A) Uses loops B) Uses function calls repeatedly C) Does not use conditions D) Always slower	**B**

4.7 Short Questions & Answers

#	Question	Answer
1	**What is recursion?**	Recursion is a technique where a function calls itself to solve a smaller sub-problem. It requires a base case to terminate.
2	**What is the base condition in recursive Binary Search?**	Two base conditions: (1) first > last → return -1 (key not found), (2) a[mid] = key → return mid+1 (key found).
3	**What is a recurrence relation?**	A recurrence relation expresses the time complexity of a recursive algorithm in terms of smaller inputs. For Binary Search: T(n) = T(n/2) + c.
4	**Why does recursive Binary Search have O(log n) space?**	Each recursive call adds a new frame to the call stack. Since there are at most $\log_2 n$ recursive calls, the maximum stack depth is $\log_2 n$, giving O(log n) space.
5	**What is the difference between actual and formal parameters?**	Actual parameters are the values passed at the point of function call. Formal parameters are the variables defined in the function header that receive those values.

4.8 Numericals

#	Problem	Solution
1	**Solve the recurrence T(n) = T(n/2) + 1 to find the time complexity of recursive Binary Search.**	T(n) = T(n/2) + 1 = T(n/4) + 1 + 1 = T(n/4) + 2 ➔ = T(n/8) + 3 ➔ = T(n/2^k) + k Stop when 2^k = n → k = $\log_2 n$ T(n) = T(1) + $\log_2 n$ = 1 + $\log_2 n$ ∴ T(n) = O(log n)
2	**For n = 128, what is the maximum recursion depth for Binary Search?**	Maximum depth = $\log_2(n)$ = $\log_2(128)$ 128 = 2^7 → $\log_2(128)$ = 7. Space used for call stack = 7 frames Answer: Depth = 7, Space = O(log 128) = O(7) = O(log n)
3	**Trace recursive Binary Search on [1, 2, 3, 4, 5, 6, 7, 8] for key = 1.**	Call 1: first=0, last=7, mid=3, a[3]=4. 1 < 4 → search left Call 2: first=0, last=2, mid=1, a[1]=2. 1 < 2 → search left Call 3: first=0, last=0, mid=0, a[0]=1. 1 = 1 → return 0+1 = 1 Answer: Found at position 1 in 3 recursive calls. Worst case $\log_2(8)$ = 3 ✓



```cpp
/*AIM- To implement the Binary Search algorithm using a recursive app
using recurrence relations.*/
//PROGRAMMER NAME = POONIS

#include<iostream>
using namespace std;

void fillArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << "Enter " << i << " Element : ";
        cin >> arr[i];

        if(i > 0 && arr[i] < arr[i - 1]){
            cout << "Data is Unsorted. Enter a value greater than or
                 << arr[i - 1] << endl;
            i--;
        }
    }
}

void displayArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << arr[i] << " ";
    }
}


int binarySearch(int arr[], int first, int last, int key){


    if(first > last){
        return -1;
    }

    int mid = first + (last - first) / 2;
    if(arr[mid] == key){
        return mid;
    }

    if(key < arr[mid]){
        return binarySearch(arr, first, mid - 1, key);
    }
    return binarySearch(arr, mid + 1, last, key);
}

int main(){
    int size, key;

    cout << "Enter Size of Array : ";
    cin >> size;

    int arr[size];

    fillArray(arr, size);

    cout << "Array Elements : ";
    displayArray(arr, size);
    cout << endl;

    cout << "Enter Key Element : ";
    cin >> key;

    int result = binarySearch(arr, 0, size - 1, key);

    if(result != -1){
        cout << "Key Found at pos : " << result+1;
    }
    else{
        cout << "Key Not Found";
    }

    return 0;
}
```

```
PS C:\out> C:\out\BinarySearch.exe
Enter Size of Array : 5
Enter 0 Element : 1
Enter 1 Element : 2
Enter 2 Element : 3
Enter 3 Element : 4
Enter 4 Element : 5
Array Elements : 1 2 3 4 5
Enter Key Element : 4
Key Found at pos : 4
PS C:\out> C:\out\BinarySearch.exe
Enter Size of Array : 5
Enter 0 Element : 1
Enter 1 Element : 2
Enter 2 Element : 3
Enter 3 Element : 4
Enter 4 Element : 5
Array Elements : 1 2 3 4 5
Enter Key Element : 12
Key Not Found
```

PROGRAM 5

Heap Sort

5.1 Aim

To implement Heap Sort using the Max-Heap data structure in C/C++, demonstrate Build-Max-Heap and Heapify operations, and analyze the time and space complexity of the algorithm.

5.2 Theory

What is a Heap?

A Heap is a complete binary tree that satisfies the heap property. It is typically stored in an array without using explicit pointers. Being a complete binary tree means every level is fully filled except possibly the last level, which is filled from left to right.

Property	Description
Complete Binary Tree	All levels filled; last level filled left to right.
Array Representation	Parent at index i has children at 2i+1 (left) and 2i+2 (right).
Parent Formula	Parent of node i = floor((i-1) / 2)
Height	O(log n) for n elements — enables efficient operations.

Max-Heap and Min-Heap

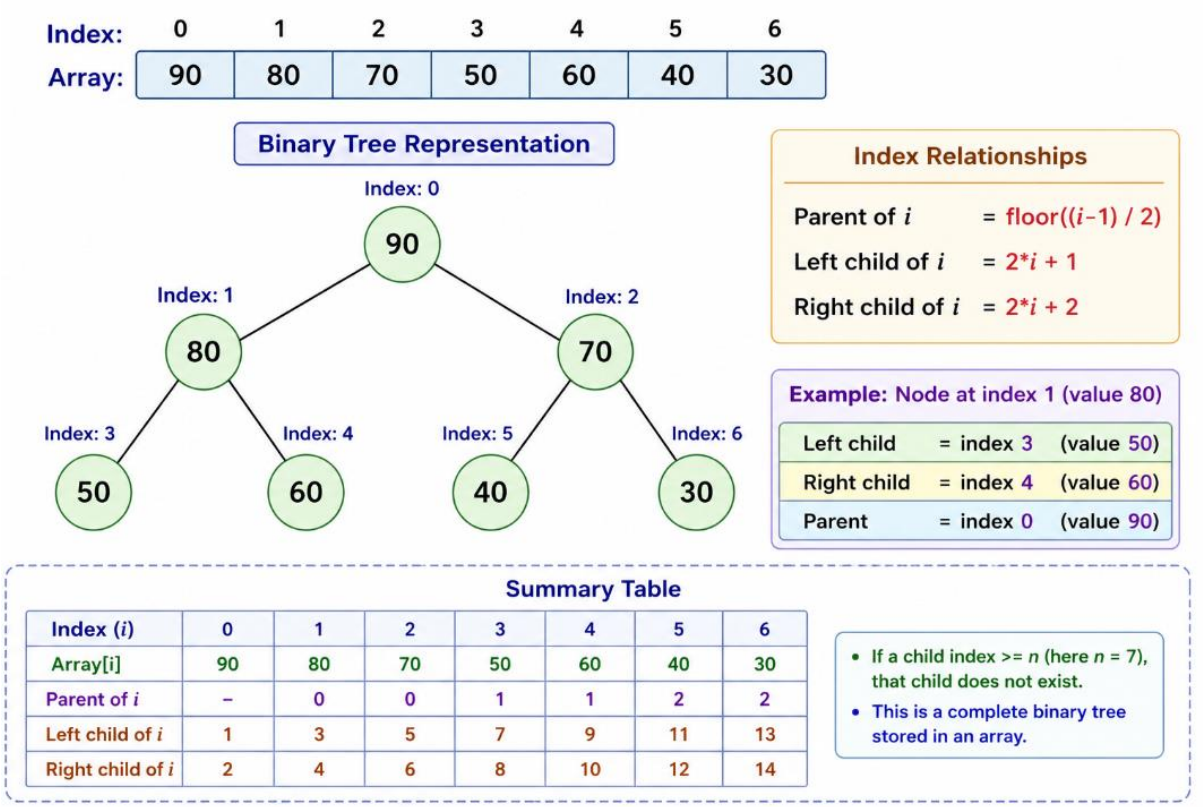
There are two variants of the heap based on the ordering property:

- Max-Heap: Every parent node is greater than or equal to its children. The root always holds the maximum element.
- Min-Heap: Every parent node is less than or equal to its children. The root always holds the minimum element.

Aspect	Max-Heap vs Min-Heap
Heap Property	Parent >= Children | Parent <= Children
Root Element	Maximum value | Minimum value
Array Example	[90, 80, 70, 50, 60] | [10, 20, 30, 40, 50]
Use Case	Heap Sort, Priority Queue (max) | Dijkstra, Prim, Priority Queue (min)

Array Representation of a Max-Heap

Max-Heap [90, 80, 70, 50, 60, 40, 30] stored as:



Heapify Operation

Heapify (also called Max-Heapify) is the key subroutine that maintains the Max-Heap property for a subtree rooted at index i, assuming both subtrees are already valid max-heaps. It runs in O(log n) time.

Key insight: Heapify only fixes a single violation — it assumes the left and right subtrees are already heaps. It compares the node with its children, swaps with the largest if needed, and recurses downward.

Build-Max-Heap

Build-Max-Heap converts an arbitrary array into a valid Max-Heap by calling Heapify on every non-leaf node, starting from index floor(n/2) - 1 down to index 0. Although it calls Heapify O(n) times, the overall complexity is O(n) — not O(n log n) — due to most nodes being near the bottom of the tree.

Heap Sort Algorithm

Heap Sort uses the Max-Heap to sort an array in ascending order. It has two phases:

- Phase 1 — Build-Max-Heap: Rearrange the array into a valid Max-Heap.
- Phase 2 — Extraction: Repeatedly swap the root (maximum) with the last element, reduce heap size by 1, and call Heapify to restore the heap property.

Property	Heap Sort
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)
Space Complexity	O(1) — In-place sorting
Stability	Not stable — relative order of equal elements may change
Adaptability	Not adaptive — always O(n log n) regardless of input order

5.3 Algorithm

HEAPIFY(A, n, i)

```
Algorithm HEAPIFY(A, n, i)
// Maintains Max-Heap property at index i (subtrees must already be heaps)
Step 1: largest ← i
      left   ← 2*i + 1
      right  ← 2*i + 2
Step 2: If left < n AND A[left] > A[largest]
          Then largest ← left
      End If
Step 3: If right < n AND A[right] > A[largest]
          Then largest ← right
      End If
Step 4: If largest ≠ i
          Then swap(A[i], A[largest])
              Call HEAPIFY(A, n, largest)   // recurse down
      End If
Step 5: Exit
```

BUILD-MAX-HEAP(A, n)

```
Algorithm BUILD-MAX-HEAP(A, n)
// Converts arbitrary array A of size n into a Max-Heap
Step 1: For i ← floor(n/2) - 1 downto 0 do
          Call HEAPIFY(A, n, i)
      End For
Step 2: Exit
```

HEAP-SORT(A, n)

```
Algorithm HEAP-SORT(A, n)
// Sorts array A of size n in ascending order using Heap Sort
Step 1: Call BUILD-MAX-HEAP(A, n)      // Phase 1: build the heap
Step 2: For i ← n-1 downto 1 do       // Phase 2: extract elements
          swap(A[0], A[i])          // move current max to end
          Call HEAPIFY(A, i, 0)      // restore heap for size i
      End For
Step 3: Exit
```

5.4 Sample Output

Test Case 1	Test Case 2
Enter number of elements: 5 Enter 5 elements: 4 10 3 5 1 Original Array: 4 10 3 5 1 Sorted Array: 1 3 4 5 10	Enter number of elements: 7 Enter 7 elements: 90 80 70 50 60 40 30 Original Array: 90 80 70 50 60 40 30 Sorted Array: 30 40 50 60 70 80 90

5.5 Time & Space Complexity

Operation	Time Complexity	Explanation
Heapify	O(log n)	Tree height; at most log n swaps downward.
Build Max-Heap	O(n)	Tighter bound due to nodes near bottom of tree.
Heap Sort (Best)	O(n log n)	n extractions, each followed by O(log n) heapify.
Heap Sort (Avg)	O(n log n)	Consistent regardless of input distribution.
Heap Sort (Worst)	O(n log n)	No degenerate case — always O(n log n).
Space Complexity	O(1)	In-place; only a constant number of extra variables.

Note: Although Build-Max-Heap calls Heapify O(n/2) times, most calls are on subtrees of height 1 or 2.
A mathematical summation shows the total work is O(n), not O(n log n).
This makes Build-Max-Heap much faster than inserting n elements one by one (which would be O(n log n)).

5.6 MCQs

#	Question	Options	Answer
1	What is a Heap?	A) Linked list B) Complete binary tree with heap property C) BST D) Graph	B
2	In a Max-Heap, the root element is always:	A) Minimum B) Maximum C) Median D) Any value	B
3	For an element at index i, the left child is at:	A) 2i B) 2i+1 C) 2i+2 D) (i-1)/2	B
4	Time complexity of Heapify operation is:	A) O(n) B) O(1) C) $O(n^2)$ D) O(log n)	D
5	Time complexity of Build-Max-Heap is:	A) O(n log n) B) O(n) C) $O(n^2)$ D) O(log n)	B
6	Time complexity of Heap Sort in all cases is:	A) $O(n^2)$ B) O(n) C) O(n log n) D) O(log n)	C
7	Space complexity of Heap Sort is:	A) O(n) B) O(n log n) C) O(log n) D) O(1)	D
8	Is Heap Sort a stable sorting algorithm?	A) Yes, always B) No, not stable C) Depends on input D) Only for integers	B
9	The starting index for Build-Max-Heap is:	A) n-1 B) 0 C) n/2 D) floor(n/2)-1	D
10	Heap Sort is most advantageous over Selection Sort because:	A) It uses less memory B) It has better time complexity C) It is stable D) It requires sorted input	B

5.7 Short Questions & Answers

#	Question	Answer
Q8	What is a Heap? Explain Max-Heap and Min-Heap with examples.	A Heap is a complete binary tree stored in an array satisfying the heap property. Max-Heap: every parent >= children; root is maximum (e.g., [90,80,70,50,60]). Min-Heap: every parent <= children; root is minimum (e.g., [10,20,30,40,50]).
Q9	What is the time complexity of Heap Sort? Why is Build-Max-Heap O(n) not O(n log n)?	Heap Sort is O(n log n) in all cases. Build-Max-Heap is O(n) because most Heapify calls are on subtrees of height 1 or 2 near the bottom; the total work sums to O(n) by mathematical analysis.

Q10	Why is Heap Sort better than Selection Sort?	Heap Sort is O(n log n) in all cases while Selection Sort is O(n^2). For large inputs (n=10,000), Heap Sort makes ~130,000 operations vs Selection Sort's 100,000,000 — making it vastly faster.
Q11	Is Heap Sort an in-place algorithm? Is it stable?	Yes, Heap Sort is in-place — it sorts within the original array using only O(1) extra space. No, it is not stable — the relative order of equal elements may change during heap operations.

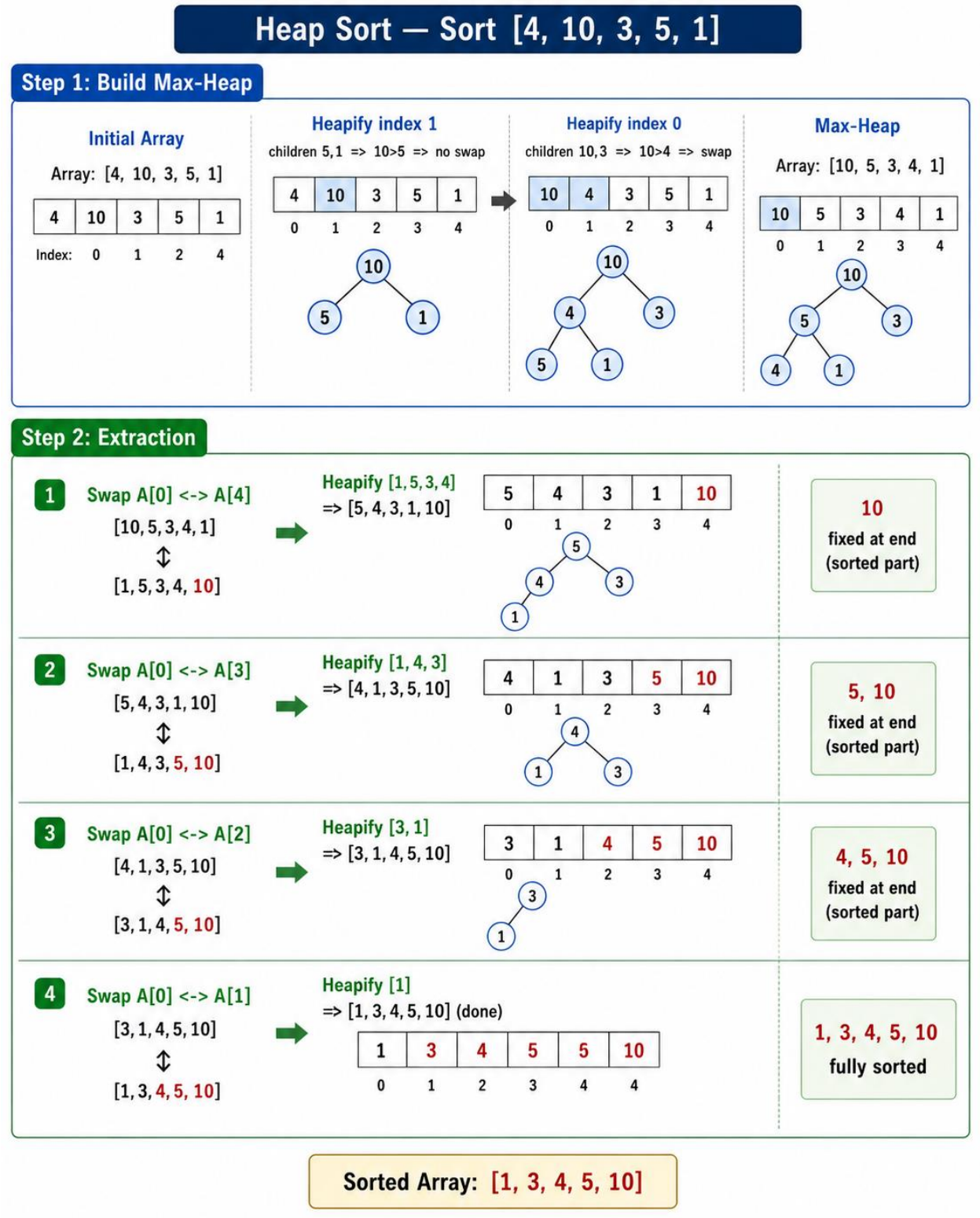
5.8 Numericals

#	Problem	Solution
1	Sort [4, 10, 3, 5, 1] using Heap Sort. Show all steps.	Step 1 — Build Max-Heap from [4, 10, 3, 5, 1] (n=5, start at index 1): Heapify(1): children=5,1; 10>5 → no swap → [4, 10, 3, 5, 1] Heapify(0): children=10,3; 10>4 → swap(0,1) → [10, 4, 3, 5, 1] Heapify(1): children=5,1; 5>4 → swap(1,3) → [10, 5, 3, 4, 1] Max-Heap: [10, 5, 3, 4, 1] Step 2 — Extraction Phase: i=4: swap(0,4) → [1,5,3,4,10]; Heapify(4,0) → [5,4,3,1,10] i=3: swap(0,3) → [1,4,3,5,10]; Heapify(3,0) → [4,1,3,5,10] i=2: swap(0,2) → [3,1,4,5,10]; Heapify(2,0) → [3,1,4,5,10] i=1: swap(0,1) → [1,3,4,5,10] Sorted Array: [1, 3, 4, 5, 10]
2	For n=8, how many times is Heapify called during Heap Sort? What is the total cost?	Build-Max-Heap calls Heapify for indices 3,2,1,0 → 4 calls = floor(8/2). Extraction phase calls Heapify 7 times (i=7 down to 1). Total Heapify calls = 4 + 7 = 11. Each Heapify is O(log 8) = O(3). Build-Max-Heap cost = O(n) = O(8). Extraction phase cost = 7 × O(log 8) = O(7 × 3) = O(21) = O(n log n).
3	Compare Heap Sort vs Selection Sort for n = 1000.	Selection Sort: O(n^2) = 1,000,000 operations. Heap Sort: O(n log n) ≈ 1000 × 10 = 10,000 operations. Heap Sort is approximately 100 times faster than Selection Sort for n=1000. Both use O(1) extra space (in-place), but Heap Sort wins significantly on time.

5.9 Heap Sort vs Other Sorting Algorithms

Algorithm	Best	Average	Worst	Space
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1) — in-place
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n) stack
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n) — not in-place
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1) — in-place
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1) — in-place

5.10. Heap Sort — Sort [4, 10, 3, 5, 1] step by step



```cpp
/*AIM- To implement Heap Sort using the Max-Heap data structure in C
operations, and analyze the time and space complexity of the algorit
//PROGRAMMER NAME = POONIS

#include<iostream>
using namespace std;

void fillArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << "Enter " << i << " Element : ";
        cin >> arr[i];
    }
}

void displayArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << arr[i] << " ";
    }
    cout << endl;
}

void heapify(int arr[], int n, int i){
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if(left < n && arr[left] > arr[largest]){
        largest = left;
    }

    if(right < n && arr[right] > arr[largest]){
        largest = right;
    }

    if(largest != i){
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void buildMaxHeap(int arr[], int n){
    for(int i = n / 2 - 1; i >= 0; i--){
        heapify(arr, n, i);
    }
}

void heapSort(int arr[], int n){
    buildMaxHeap(arr, n);

    for(int i = n - 1; i > 0; i--){
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main(){
    int size;

    cout << "Enter Size of Array : ";
    cin >> size;

    int arr[size];

    fillArray(arr, size);

    cout << "Array Elements : ";
    displayArray(arr, size);

    heapSort(arr, size);

    cout << "Sorted Array : ";
    displayArray(arr, size);

    return 0;
}
```

```
PS C:\out> C:\out\HeapSort.exe
Enter Size of Array : 5
Enter 0 Element : 4
Enter 1 Element : 10
Enter 2 Element : 3
Enter 3 Element : 5
Enter 4 Element : 1
Array Elements : 4 10 3 5 1
Sorted Array : 1 3 4 5 10
PS C:\out> C:\out\HeapSort.exe
Enter Size of Array : 7
Enter 0 Element : 90
Enter 1 Element : 80
Enter 2 Element : 70
Enter 3 Element : 50
Enter 4 Element : 60
Enter 5 Element : 40
Enter 6 Element : 30
Array Elements : 90 80 70 50 60 40 30
Sorted Array : 30 40 50 60 70 80 90
```

PROGRAM 6

Merge Sort

6.1 Aim

To implement Merge Sort using the Divide and Conquer approach and analyze its time and space complexity.

6.2 Theory

What is Merge Sort?

Merge Sort is a comparison-based sorting algorithm that follows the Divide and Conquer paradigm. It recursively divides the array into two equal halves until each subarray has a single element (which is trivially sorted), and then merges the sorted halves back together. The key operation is the MERGE step, which combines two sorted subarrays into one sorted array.

Divide and Conquer Strategy

Phase	Description
Divide	Split the array at the midpoint: mid = (left + right) / 2. Recurse on A[left..mid] and A[mid+1..right].
Conquer	Recursively sort each half until subarrays are of size 1 (base case: left >= right).
Combine	Merge the two sorted halves using temporary arrays L[] and R[]. Compare elements from L and R, placing the smaller one back into A[].

Recurrence Relation & Master Theorem

$T(n) = 2T(n/2) + O(n)$ → Solved by Master Theorem Case 2 → $T(n) = \Theta(n \log n)$

Master Theorem Case 2: When a = b^k (here $2 = 2^1$, so k=1) and the work per level is O(n^k), the solution is $T(n) = O(n^k \log n) = O(n \log n)$.

6.3 Algorithm

MERGE-SORT(A, left, right)	**MERGE(A, left, mid, right)**

if left < right:
  mid = floor((left + right) / 2)
  MERGE-SORT(A, left, mid)
  MERGE-SORT(A, mid+1, right)
  MERGE(A, left, mid, right)
	Create temp arrays L[], R[]
	Copy A[left..mid] to L
	Copy A[mid+1..right] to R
	i = 0, j = 0, k = left
	while i < len(L) and j < len(R):
	  if L[i] <= R[j]: A[k++] = L[i++]
	  else: A[k++] = R[j++]
	Copy remaining L and R to A

6.4 Time & Space Complexity

Case	Recurrence	Time Complexity	Space Complexity
Best Case	T(n) = 2T(n/2) + O(n)	O(n log n)	O(n)
Average Case	T(n) = 2T(n/2) + O(n)	O(n log n)	O(n)
Worst Case	T(n) = 2T(n/2) + O(n)	O(n log n)	O(n)

6.5 Sample Output (Trace)

#	Problem	Solution
Q21	**Sort [38, 27, 43, 3, 9, 82, 10] using Merge Sort. Show all levels.**	Level 0 (original): [38, 27, 43, 3, 9, 82, 10] Level 1 (divide): [38,27,43] [3,9,82,10] Level 2 (divide): [38][27,43] [3,9][82,10] Level 3 (divide): [38][27][43] [3][9][82][10] Level 2 (merge): [38][27,43] [3,9][10,82] Level 1 (merge): [27,38,43] [3,9,10,82] Level 0 (merge): [3,9,10,27,38,43,82] **Sorted: [3, 9, 10, 27, 38, 43, 82]**

6.6 MCQs

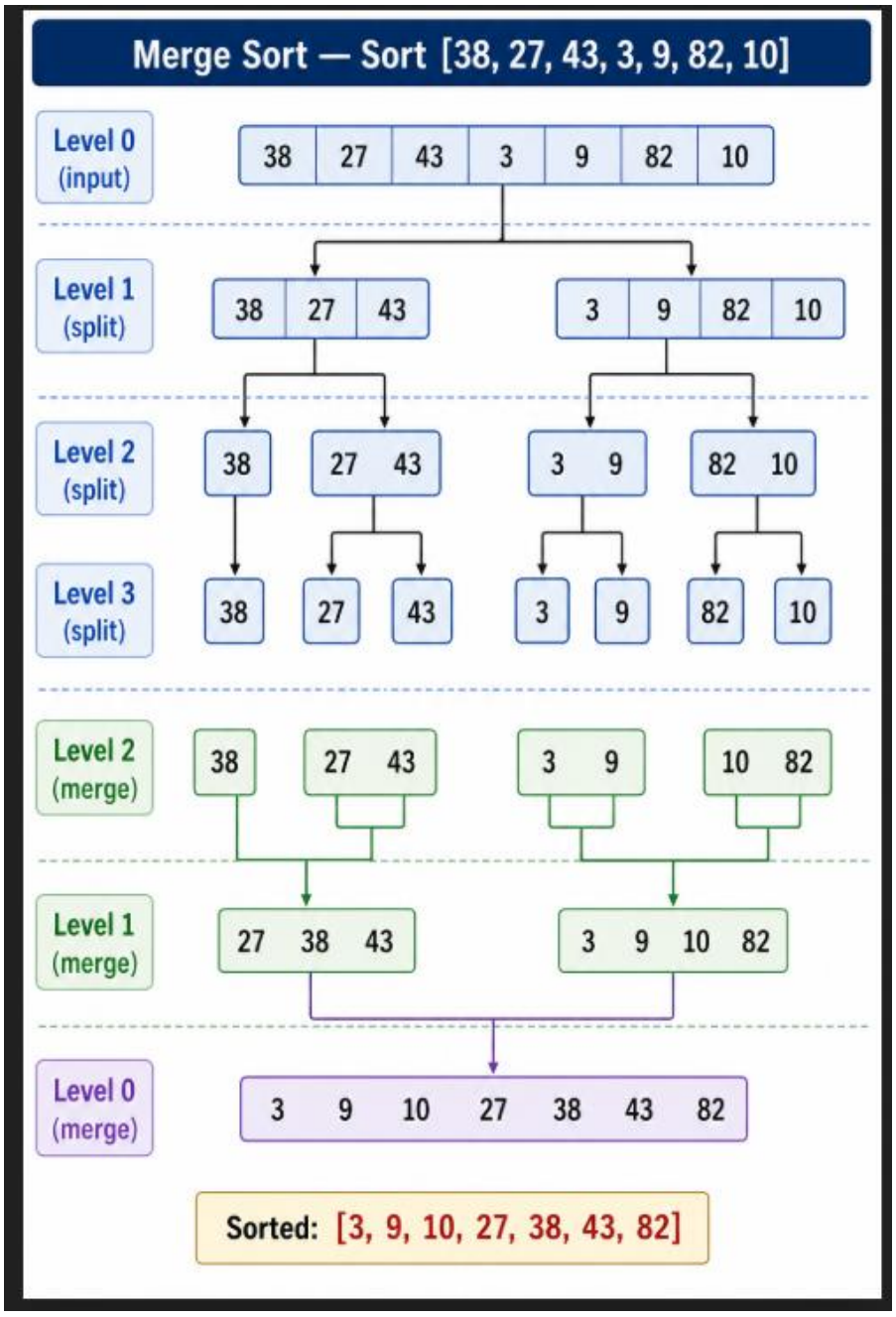
#	Question	Options	Answer
1	What is the time complexity of Merge Sort in all cases?	A) O(n) B) O(n^2) C) O(n log n) D) O(log n)	**C**
2	What technique does Merge Sort use?	A) Greedy B) Dynamic Programming C) Backtracking D) Divide and Conquer	**D**
3	What is the space complexity of Merge Sort?	A) O(1) B) O(log n) C) O(n) D) O(n^2)	**C**
4	Merge Sort is called stable because:	A) It is fast B) Equal elements maintain their original relative order C) It uses recursion D) No extra memory is used	**B**
5	In Merge Sort, when does recursion stop?	A) When array is sorted B) When left >= right C) When mid = 0 D) When array has no elements	**B**
6	What does the MERGE step do?	A) Splits array B) Compares pivot C) Merges two sorted halves D) Swaps elements	**C**
7	Recurrence relation for Merge Sort is:	A) T(n) = T(n/2) + O(1) B) T(n) = 2T(n/2) + O(n) C) T(n) = T(n-1) + O(n) D) T(n) = nT(n)	**B**
8	Which Master Theorem case solves Merge Sort?	A) Case 1 B) Case 2 C) Case 3 D) None	**B**
9	Merge Sort is best suited for:	A) Arrays with few elements B) Linked lists and external sorting C) Almost sorted arrays D) Random access memory only	**B**
10	What are the auxiliary arrays in Merge Sort called?	A) Buffer arrays B) Temp left L[] and right R[] arrays C) Pivot arrays D) Recursive arrays	**B**

6.7 Short Questions & Answers

#	Question	Answer
1	**What is Merge Sort?**	Merge Sort is a Divide and Conquer sorting algorithm. It divides the array into two halves, recursively sorts each half, and then merges the two sorted halves into a single sorted array.
2	**Why is Merge Sort's worst case O(n log n) unlike Quick Sort?**	Because Merge Sort always divides the array into exactly two equal halves regardless of the data. The depth of recursion is always log n, and each level does O(n) work in the merge step.
3	**What is the significance of the MERGE step?**	The MERGE step combines two sorted subarrays into one sorted array by comparing elements one by one and placing the smaller element first.
4	**Why does Merge Sort need O(n) extra space?**	It requires temporary arrays L[] and R[] to hold elements during the merge process. These cannot be done in-place efficiently, requiring extra space proportional to the array size.
5	**What is the Master Theorem Case 2?**	When a = b^k and f(n) = O(n^k log^p n), the solution is T(n) = O(n^k log^(p+1) n). For Merge Sort: a=2, b=2, k=1, p=0, so T(n) = O(n log n).

6	**Why is Merge Sort preferred over Quick Sort in certain cases? (Q48)**	• Worst Case Guarantee: Merge Sort is always O(n log n); Quick Sort degrades to O(n^2) on sorted/reverse-sorted input. • Stability: Merge Sort is stable (preserves relative order of equal elements); Quick Sort is generally not stable. • External Sorting: Merge Sort is ideal for data on disk (sequential access); Quick Sort needs random access. • Linked Lists: Merge Sort works efficiently on linked lists; Quick Sort performs poorly due to non-random access. Predictable Performance: Merge Sort's performance is predictable; Quick Sort depends heavily on pivot selection.

Merge Sort — Sort [38, 27, 43, 3, 9, 82, 10]



```cpp
//AIM- To implement Merge Sort using the Divide and Conquer approach
//PROGRAMMER NAME = POONIS

#include<iostream>
using namespace std;

void fillArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << "Enter " << i << " Element : ";
        cin >> arr[i];
    }
}

void displayArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << arr[i] << " ";
    }
    cout << endl;
}

void merge(int arr[], int left, int mid, int right){
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for(int i = 0; i < n1; i++){
        L[i] = arr[left + i];
    }

    for(int j = 0; j < n2; j++){
        R[j] = arr[mid + 1 + j];
    }

    int i = 0, j = 0, k = left;

    while(i < n1 && j < n2){
        if(L[i] <= R[j]){
            arr[k] = L[i];
            i++;
        }
        else{
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while(i < n1){
        arr[k] = L[i];
        i++;
        k++;
    }

    while(j < n2){
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int left, int right){
    if(left < right){
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main(){
    int size;

    cout << "Enter Size of Array : ";
    cin >> size;

    int arr[size];

    fillArray(arr, size);

    cout << "Array Elements : ";
    displayArray(arr, size);

    mergeSort(arr, 0, size - 1);

    cout << "Sorted Array : ";
    displayArray(arr, size);

    return 0;
}
```

```
PS C:\out> C:\out\MergeSort.exe
Enter Size of Array : 7
Enter 0 Element : 38
Enter 1 Element : 27
Enter 2 Element : 43
Enter 3 Element : 3
Enter 4 Element : 9
Enter 5 Element : 82
Enter 6 Element : 10
Array Elements : 38 27 43 3 9 82 10
Sorted Array : 3 9 10 27 38 43 82
```

PROGRAM 7

Quick Sort

7.1 Aim

To implement Quick Sort using the Divide and Conquer approach with Lomuto partitioning and analyze its time and space complexity.

7.2 Theory

What is Quick Sort?

Quick Sort is an in-place, comparison-based sorting algorithm that uses Divide and Conquer. It selects a pivot element, partitions the array so elements smaller than or equal to the pivot are on the left and elements greater are on the right, then recursively sorts each partition. Unlike Merge Sort, Quick Sort does not require extra arrays — the partitioning is done in place.

Divide and Conquer Strategy

Phase	Description
Divide	Choose pivot (last element). Call PARTITION to place pivot at correct index pi. Array is divided at pi.
Conquer	Recursively apply Quick Sort on A[low..pi-1] (left of pivot) and A[pi+1..high] (right of pivot).
Combine	No explicit merge step needed — after recursion, the array is sorted in place. The pivot is already in its final position.

Recurrence Relations

Case	Recurrence	Solution
Best/Average	T(n) = 2T(n/2) + O(n) (balanced pivot)	O(n log n)
Worst	T(n) = T(n-1) + O(n) (most unbalanced pivot)	$O(n^2)$

7.3 Algorithm

QUICK-SORT(A, low, high)	PARTITION(A, low, high) — Lomuto Scheme

if low < high:
  pi = PARTITION(A, low, high)
  QUICK-SORT(A, low, pi - 1)
  QUICK-SORT(A, pi + 1, high)
	pivot = A[high]
	i = low - 1
	for j = low to high - 1:
	  if A[j] <= pivot:
	  i++
	  swap(A[i], A[j])
	swap(A[i+1], A[high])
	return i + 1

7.4 Time & Space Complexity

Case	Condition	Recurrence	Time	Space
Best Case	Pivot always at middle	T(n) = 2T(n/2)+O(n)	O(n log n)	O(log n)
Average Case	Random pivot	T(n) = O(n log n) avg	O(n log n)	O(log n)
Worst Case	Sorted/reverse sorted	T(n) = T(n-1)+O(n)	$O(n^2)$	O(n)

7.5 Sample Output (Trace)

#	Problem	Solution
Q23	**Sort [10, 80, 30, 90, 40, 50, 70] using Quick Sort (pivot = last element). Show step-by-step partition.**	Initial: [10, 80, 30, 90, 40, 50, 70], pivot=70 After partition: [10, 30, 40, 50, 70, 90, 80] pivot at index 4 Left subarray [10, 80, 30, 90, 40, 50], pivot=50 After partition: [10, 30, 40, 50, 90, 80] pivot at index 3 Left [10, 80, 30, 40], pivot=40 -> [10, 30, 40, 80] Left [10, 30], pivot=30 -> [10, 30] pivot at index 1 Right [80] -> trivially sorted Right subarray [90, 80], pivot=80 -> [80, 90] **Final sorted: [10, 30, 40, 50, 70, 80, 90]**

7.6 MCQs

#	Question	Options	Answer
1	What is the worst-case time complexity of Quick Sort?	A) O(n log n) B) O(n) C) $O(n^2)$ D) O(log n)	**C**
2	Worst case of Quick Sort occurs when:	A) Pivot is always at the middle B) Array is random C) Pivot is always smallest or largest element D) Array has unique elements	**C**
3	What is the best-case time complexity of Quick Sort?	A) $O(n^2)$ B) O(n log n) C) O(n) D) O(log n)	**B**
4	In Lomuto partition, which element is chosen as pivot?	A) First element B) Middle element C) Last element D) Random element	**C**
5	Quick Sort is an in-place algorithm because:	A) It needs O(n) space B) It only needs O(log n) auxiliary stack space C) It uses linked lists D) It copies the array	**B**
6	Quick Sort is NOT stable because:	A) It is recursive B) Long-distance swaps can change relative order of equal elements C) It uses a pivot D) It is too fast	**B**
7	Recurrence for Quick Sort worst case is:	A) T(n) = 2T(n/2) + O(n) B) T(n) = T(n-1) + O(n) C) T(n) = T(n/2) + O(1) D) T(n) = T(n+1) + O(n)	**B**
8	Which variable tracks the position boundary in Lomuto partition?	A) j B) pivot C) i D) k	**C**
9	Quick Sort is preferred over Merge Sort in practice because:	A) Better worst case B) It is stable C) Cache friendly and no extra memory D) It is non-recursive	**C**
10	After PARTITION, the pivot element is:	A) At position 0 B) In its final sorted position C) At the end D) At the midpoint	**B**

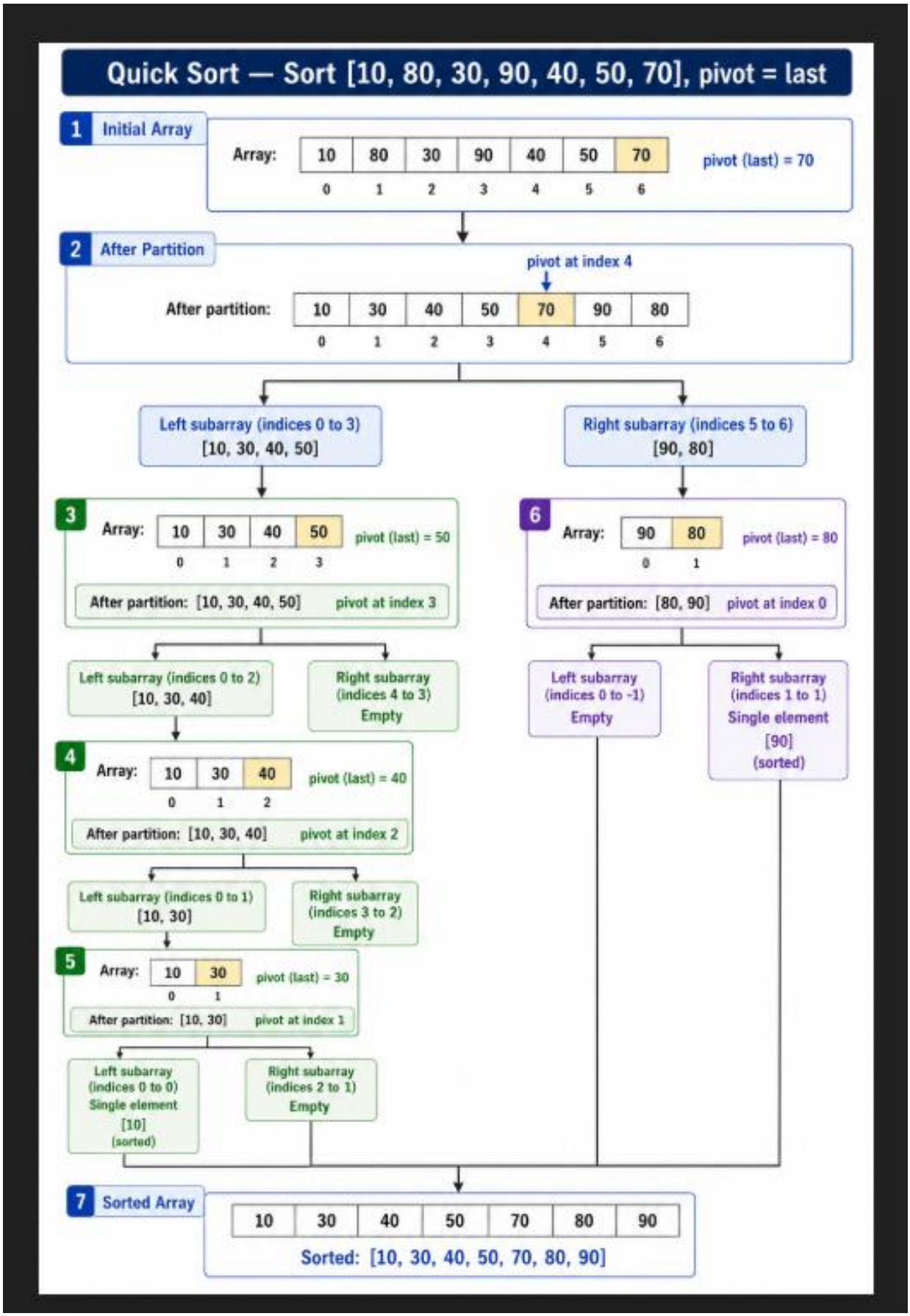
7.7 Short Questions & Answers

#	Question	Answer
1	**What is Quick Sort?**	Quick Sort is a Divide and Conquer sorting algorithm. It selects a pivot element, partitions the array around the pivot (elements smaller to the left, larger to the right), and recursively sorts the two subarrays.
2	**What is the role of the PARTITION function?**	The PARTITION function rearranges the array so that all elements less than or equal to the pivot come before it, and all elements greater come after it. The pivot is then placed at its correct sorted position.
3	**Why is Quick Sort's worst case $O(n^2)$?**	When the pivot is always the smallest or largest element (e.g., sorted array with last element as pivot), the partition produces one subarray of size n-1 and one of size 0, leading to $T(n) = T(n-1) + O(n) \rightarrow O(n^2)$.
4	**What is the Lomuto partition scheme?**	In Lomuto, the last element is chosen as pivot. A boundary index i tracks elements ≤ pivot. For each element ≤ pivot, i is incremented and elements are swapped. Finally, the pivot is swapped to position i+1.
5	**How can Quick Sort's worst case be avoided?**	By using randomized pivot selection (pick a random element as pivot) or the median-of-three method (choose the median of first, middle, and last elements). This makes worst-case behaviour extremely unlikely.

7.8 Quick Sort vs Merge Sort

Aspect	Quick Sort	Merge Sort
Strategy	Partition-based (in-place)	Divide and merge (uses extra arrays)
Best Case	O(n log n)	O(n log n)
Average Case	O(n log n)	O(n log n)
Worst Case	$O(n^2)$ — sorted/reverse sorted input	O(n log n) — always
Space Complexity	O(log n) average, O(n) worst (stack)	O(n) — always needs extra arrays
Stability	Not stable (swaps may disrupt equal elements)	Stable (preserves relative order of equal elements)
Practical Speed	Faster in practice (cache-friendly, less overhead)	Slower due to extra memory allocation
Best Used For	In-memory sorting, small to large data sets	Linked lists, external sorting, stable sort needed
Adaptiveness	Can be randomized to avoid worst case	Performance is always predictable

7.9 NumericalQuick Sort — Sort [10, 80, 30, 90, 40, 50, 70], pivot=last


Quick Sort — Sort [10, 80, 30, 90, 40, 50, 70], pivot = last
1 Initial Array
Array: 10 80 30 90 40 50 70
0 1 2 3 4 5 6
pivot (last) = 70
2 After Partition
pivot at index 4
After partition: 10 30 40 50 70 90 80
0 1 2 3 4 5 6
Left subarray (indices 0 to 3)
[10, 30, 40, 50]
Right subarray (indices 5 to 6)
[90, 80]
3
Array: 10 30 40 50
0 1 2 3
pivot (last) = 50
After partition: [10, 30, 40, 50] pivot at index 3
Left subarray (indices 0 to 2)
[10, 30, 40]
Right subarray
(indices 4 to 3)
Empty
4
Array: 10 30 40
0 1 2
pivot (last) = 40
After partition: [10, 30, 40] pivot at index 2
Left subarray (indices 0 to 1)
[10, 30]
Right subarray
(indices 3 to 2)
Empty
5
Array: 10 30
0 1
pivot (last) = 30
After partition: [10, 30] pivot at index 1
Left subarray
(indices 0 to 0)
Single element
[10]
(sorted)
Right subarray
(indices 2 to 1)
Empty
6
Array: 90 80
0 1
pivot (last) = 80
After partition: [80, 90] pivot at index 0
Left subarray
(indices 0 to -1)
Empty
Right subarray
(indices 1 to 1)
Single element
[90]
(sorted)
7 Sorted Array
10 30 40 50 70 80 90
Sorted: [10, 30, 40, 50, 70, 80, 90]

```cpp
/* AIM- To implement Quick Sort using the Divide and Conquer approach
complexity.*/
//PROGRAMMER NAME = POONIS

#include<iostream>
using namespace std;

void fillArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << "Enter " << i << " Element : ";
        cin >> arr[i];
    }
}

void displayArray(int arr[], int size){
    for(int i = 0; i < size; i++){
        cout << arr[i] << " ";
    }
    cout << endl;
}

int partition(int arr[], int low, int high){
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j <= high - 1; j++){
        if(arr[j] <= pivot){
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);

    return i + 1;
}

void quickSort(int arr[], int low, int high){
    if(low < high){
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main(){
    int size;

    cout << "Enter Size of Array : ";
    cin >> size;

    int arr[size];

    fillArray(arr, size);

    cout << "Array Elements : ";
    displayArray(arr, size);

    quickSort(arr, 0, size - 1);

    cout << "Sorted Array : ";
    displayArray(arr, size);

    return 0;
}
```

```
PS C:\out> C:\out\QuickSort.exe
Enter Size of Array : 7
Enter 0 Element : 10
Enter 1 Element : 80
Enter 2 Element : 30
Enter 3 Element : 90
Enter 4 Element : 40
Enter 5 Element : 50
Enter 6 Element : 70
Array Elements : 10 80 30 90 40 50 70
Sorted Array : 10 30 40 50 70 80 90
```

PROGRAM 8

X*Y, X^Y, X/Y using Recursion

8.1 Aim

To implement arithmetic operations — Multiplication (X×Y), Exponentiation (X^Y), and Integer Division (X÷Y) — using Recursion in C/C++, understand the concept of base case and recursive case, and analyze the time and space complexity of each recursive function.

8.2 Theory

What is Recursion?

Recursion is a problem-solving technique in which a function calls itself with a smaller or simpler version of the original problem. Each recursive call brings the problem closer to a base case, at which point the recursion stops and results are returned back up the call chain. Recursion is powerful for tasks that can be naturally broken into identical sub-problems.

Component	Description
Base Case	The simplest version of the problem that can be solved directly without further recursion. Without it, recursion never stops (stack overflow).
Recursive Case	The part of the function where it calls itself with a reduced problem. Must always move towards the base case.
Call Stack	Each function call pushes a stack frame onto memory. Recursion depth = number of stack frames used simultaneously.
Return Phase	Once the base case is reached, results propagate back up through all the stack frames (unwinding the call stack).

Recursive Arithmetic — Core Ideas

Operation	Math Insight	Recursive Definition	Base Case
X * Y	Multiplication is repeated addition	MULTIPLY(X,Y) = X + MULTIPLY(X, Y-1)	MULTIPLY(X, 0) = 0
X ^ Y	Exponentiation is repeated multiplication	POWER(X,Y) = X × POWER(X, Y-1)	POWER(X, 0) = 1
X / Y	Integer division via repeated subtraction	DIVIDE(X,Y) = 1 + DIVIDE(X-Y, Y)	DIVIDE(X,Y) = 0 if X < Y

Recurrence Relations

Each operation has a recurrence that describes how the total work grows with input:

MULTIPLY(X,Y): $T(Y) = T(Y-1) + O(1) \rightarrow T(Y) = O(Y)$
POWER(X,Y): $T(Y) = T(Y-1) + O(1) \rightarrow T(Y) = O(Y)$
DIVIDE(X,Y): $T(X) = T(X-Y) + O(1) \rightarrow T(X) = O(X/Y)$ (integer quotient steps)

8.3 Algorithm

MULTIPLY(X, Y)	POWER(X, Y)	DIVIDE(X, Y)

// Returns X * Y recursively
// using repeated addition
MULTIPLY(X, Y):
  if Y = 0:
  return 0
  if Y < 0:
  return -MULTIPLY(X, -Y)
  return X + MULTIPLY(X, Y-1)
 	
// Base: MULTIPLY(X,0)=0
// Step: X + X*(Y-1)
	// Returns X ^ Y recursively
	// (X to the power Y)
	POWER(X, Y):
	  if Y = 0:
	  return 1
	  if Y < 0:
	  return 1/POWER(X,-Y)
	  return X * POWER(X, Y-1)
	
	// Base: POWER(X,0)=1
	// Step: X * X^(Y-1)
		// Returns X / Y recursively
		// via repeated subtraction
		DIVIDE(X, Y):
		  if Y = 0:
		  error: division by zero
		  if X < Y:
		  return 0
		  return 1 + DIVIDE(X-Y, Y)
		
		// Base: X < Y → 0
		// Step: 1 + (X-Y)/Y

Main Algorithm — Main()

```
Algorithm Main()
Step 1: Read X and Y
Step 2: result_mul ← MULTIPLY(X, Y)
      Print 'X * Y =', result_mul
Step 3: result_pow ← POWER(X, Y)
      Print 'X ^ Y =', result_pow
Step 4: If Y = 0
         Print 'Division by zero not allowed'
      Else
         result_div ← DIVIDE(X, Y)
         Print 'X / Y =', result_div
Step 5: Exit
```

8.4 Sample Output

Test Case 1 — X = 4, Y = 3

```
Enter X: 4
Enter Y: 3
4 * 3 = 12
4 ^ 3 = 64
4 / 3 = 1
```

Test Case 2 — X = 10, Y = 2

```
Enter X: 10
Enter Y: 2
10 * 2 = 20
10 ^ 2 = 100
10 / 2 = 5
```

Test Case 3 — X = 5, Y = 0 (edge case)

```
Enter X: 5
Enter Y: 0
5 * 0 = 0
5 ^ 0 = 1
Division by zero not allowed
```

8.5 Time & Space Complexity

Operation	Base Case	Recurrence	Time Complexity	Space (Stack)	Result
X * Y	Y = 0 → 0	T(Y) = T(Y-1)+O(1)	O(Y)	O(Y)	X+X+…(Y times)
X ^ Y	Y = 0 → 1	T(Y) = T(Y-1)+O(1)	O(Y)	O(Y)	X×X×…(Y times)
X / Y	X < Y → 0	T(X) = T(X-Y)+O(1)	O(X/Y)	O(X/Y)	Quotient only

Note: Space complexity refers to the call stack depth (auxiliary space). No input arrays are used — all three operations use O(depth) stack space where depth equals the number of recursive calls made.

8.6 MCQs

#	Question	Options	Ans
1	What is the base case for recursive multiplication MULTIPLY(X,Y)?	A) Y=1 → X B) Y=0 → 0 C) X=0 → Y D) Y=0 → 1	**B**
2	Which concept does recursive X*Y use internally?	A) Binary search B) Subtraction C) Repeated addition D) Division	**C**
3	What is the base case for recursive POWER(X,Y)?	A) Y=1 → X B) X=0 → 0 C) Y=0 → 1 D) X=1 → 1	**C**
4	Recurrence for POWER(X,Y) is:	A) T(Y)=2T(Y/2)+O(1) B) T(Y)=T(Y-1)+O(1) C) T(Y)=T(Y)+O(n) D) T(Y)=O(1)	**B**
5	What does recursive DIVIDE(X,Y) compute?	A) Exact quotient with remainder B) Integer quotient via repeated subtraction C) Floating-point result D) GCD of X and Y	**B**
6	Time complexity of MULTIPLY(X,Y) using recursion is:	A) O(X) B) O(XY) C) O(Y) D) O(1)	**C**
7	What is the space complexity of recursive POWER(X,Y)?	A) O(1) B) O(log Y) C) O(X) D) O(Y)	**D**
8	What happens when Y is negative in MULTIPLY(X,Y)?	A) Returns 0 B) Infinite recursion C) Returns -MULTIPLY(X,-Y) D) Returns X	**C**
9	Which call stack depth does DIVIDE(12,4) use?	A) 4 B) 12 C) 3 D) 48	**C**
10	Why does recursion use O(Y) stack space for POWER(X,Y)?	A) Each call stores the full array B) One stack frame per recursive call, depth = Y C) Two arrays are created D) It is iterative	**B**

8.7 Short Questions & Answers

#	Question	Answer
1	**What is recursion?**	Recursion is a programming technique where a function calls itself to solve a smaller sub-problem. Every recursive function must have a base case (stopping condition) and a recursive case that reduces the problem towards the base case.
2	**What is the base case, and why is it needed?**	The base case is a condition under which the function returns a result directly without calling itself. It is essential to prevent infinite recursion. Without it, the function would keep calling itself until a stack overflow occurs.
3	**How does MULTIPLY(X,Y) work recursively?**	MULTIPLY(X,Y) adds X to itself Y times using recursion: MULTIPLY(X,Y) = X + MULTIPLY(X,Y-1), stopping when Y=0 (returns 0). For example, MULTIPLY(4,3) = 4+MULTIPLY(4,2) = 4+4+MULTIPLY(4,1) = 4+4+4+0 = 12.
4	**How does POWER(X,Y) work recursively?**	POWER(X,Y) multiplies X by itself Y times: POWER(X,Y) = X × POWER(X,Y-1), stopping when Y=0 (returns 1). For example, POWER(2,4) = 2×POWER(2,3) = 2×2×2×2×1 = 16.
5	**How does DIVIDE(X,Y) work recursively?**	DIVIDE(X,Y) counts how many times Y can be subtracted from X: DIVIDE(X,Y) = 1 + DIVIDE(X-Y,Y), stopping when X<Y (returns 0). For example, DIVIDE(10,3) = 1+DIVIDE(7,3) = 1+1+DIVIDE(4,3) = 1+1+1+DIVIDE(1,3) = 3.

8.8 Numericals

#	Problem	Step-by-Step Solution
N1	**Trace MULTIPLY(3, 4) step by step and count recursive calls.**	MULTIPLY(3, 4) = 3 + MULTIPLY(3, 3) [call 1] = 3 + 3 + MULTIPLY(3, 2) [call 2] = 3 + 3 + 3 + MULTIPLY(3,1)[call 3] = 3 + 3 + 3 + 3 + MULTIPLY(3,0) [call 4] = 3 + 3 + 3 + 3 + 3 + 0 [base case] = 12 Total recursive calls: 4 (= Y) Time: O(Y) = O(4) | Space: O(4) stack frames
N2	**Trace POWER(2, 5) step by step and state recurrence solution.**	POWER(2, 5) = 2 × POWER(2, 4) [call 1] = 2 × 2 × POWER(2, 3) [call 2] = 2 × 2 × 2 × POWER(2, 2) [call 3] = 2 × 2 × 2 × 2 × POWER(2,1)[call 4] = 2 × 2 × 2 × 2 × 2 × POWER(2,0)[call 5] = 2 × 2 × 2 × 2 × 2 × 1 [base case] = 32 Recurrence: T(Y)=T(Y-1)+O(1) → T(Y)=O(Y) Time: O(5) | Space: O(5) stack frames
N3	**Trace DIVIDE(13, 3) and verify result.**	DIVIDE(13, 3) = 1 + DIVIDE(10, 3) [13 >= 3, subtract] = 1 + 1 + DIVIDE(7,3) [10 >= 3, subtract] = 1 + 1 + 1 + DIVIDE(4,3)[7 >= 3, subtract] = 1 + 1 + 1 + 1 + DIVIDE(1,3)[4>=3,subtract] = 1+1+1+1+0 [1 < 3, base case] = 4 **Verify: 13 ÷ 3 = 4 remainder 1 ✓** Time: O(X/Y)=O(4) | Space: O(4) stack frames
N4	**What is POWER(3,0)? What is MULTIPLY(7,0)? What is DIVIDE(5,6)?**	POWER(3, 0): Base case Y=0 → return 1 Result = 1 MULTIPLY(7, 0): Base case Y=0 → return 0 Result = 0 DIVIDE(5, 6): Base case X < Y → 5 < 6 → return 0 Result = 0 (quotient, since 5/6 < 1)

8.9 Recursion vs Iteration — Comparison

Aspect	Recursive Approach	Iterative Approach
Code Clarity	Short, elegant, mirrors math definition	Longer but explicit loop structure
Space	O(Y) stack frames — risk of stack overflow	O(1) — no call stack overhead
Speed	Function call overhead per recursion	Faster in practice — no call overhead
Base Case	Mandatory to prevent infinite recursion	Loop condition naturally stops
Debugging	Harder — must trace call stack	Easier — step through loop
Use Case	Teaching, small Y values	Production / large inputs

```cpp
/*AIM- To implement arithmetic operations – Multiplication (X×Y), Exp
Recursion in C/C++, understand the concept of base case and recursive
each recursive function.*/
//PROGRAMMER NAME = POONIS

#include<iostream>
using namespace std;

int multiply(int x, int y){
    if(y == 0){
        return 0;
    }

    if(y < 0){
        return -multiply(x, -y);
    }

    return x + multiply(x, y - 1);
}

double power(double x, int y){
    if(y == 0){
        return 1;
    }

    if(y < 0){
        return 1 / power(x, -y);
    }

    return x * power(x, y - 1);
}

int divide(int x, int y){
    if(x < y){
        return 0;
    }

    return 1 + divide(x - y, y);
}

int main(){
    int x, y;

    cout << "Enter Value of X : ";
    cin >> x;

    cout << "Enter Value of Y : ";
    cin >> y;

    int result_mul = multiply(x, y);
    cout << "X * Y = " << result_mul << endl;

    double result_pow = power(x, y);
    cout << "X ^ Y = " << result_pow << endl;

    if(y == 0){
        cout << "Division by Zero Not Allowed" << endl;
    }
    else{
        int result_div = divide(x, y);
        cout << "X / Y = " << result_div << endl;
    }

    return 0;
}
```

```
PS C:\out> C:\out\AirthmaticOperation.exe
Enter Value of X : 4
Enter Value of Y : 3
X * Y = 12
X ^ Y = 64
X / Y = 1
PS C:\out> C:\out\AirthmaticOperation.exe
Enter Value of X : 10
Enter Value of Y : 2
X * Y = 20
X ^ Y = 100
X / Y = 5
PS C:\out> C:\out\AirthmaticOperation.exe
Enter Value of X : 5
Enter Value of Y : 0
X * Y = 0
X ^ Y = 1
Division by Zero Not Allowed
```

PROGRAM 9

X*Y, X^Y, X/Y using Divide and Conquer

9.1 Aim

To implement multiplication (X*Y), exponentiation (X^Y), and division (X/Y) using the Divide and Conquer approach in C++, and analyze their time and space complexities.

9.2 Theory

Divide and Conquer is a problem-solving strategy in which a problem is divided into smaller sub-problems, solved recursively, and then combined to obtain the final result.

The three operations implemented are:

1. Multiplication (X * Y)
The multiplication is performed recursively by halving one number at each recursive call.
If X is even:
   X * Y = 2 * ((X/2) * Y)
If X is odd:
   X * Y = 2 * ((X/2) * Y) + Y

2. Power (X ^ Y)
Exponentiation uses repeated squaring.
If Y is even:
   X^Y = (X*X)^(Y/2)
If Y is odd:
   X^Y = (X*X)^((Y-1)/2) * X

3. Division (X / Y)
Division is implemented recursively using halving.
If X is even:
   X/Y = 2 * ((X/2)/Y)
If X is odd:
   X/Y = 2 * ((X/2)/Y) + 1/Y

These recursive approaches reduce the problem size at every step, leading to logarithmic time complexity.

9.3 Algorithms

Algorithm MULT(x, y) Step 1: If x = 1 return y Step 2: temp ← 2 × MULT(x/2, y) Step 3: If x is odd then temp ← temp + y Step 4: Return temp	Algorithm POWER(x, y) Step 1: If y = 1 return x Step 2: If y is even then return POWER(x*x, y/2) Else return POWER(x*x, (y-1)/2) × x	Algorithm DIV(x, y) Step 1: If x ≤ y return x/y Step 2: If x is even then return 2 × DIV(x/2, y) Else return 2 × DIV(x/2, y) + 1/y

9.7 Sample Output

Test Case 1 – Multiplication Enter two numbers to Multiply : 10 3 Answer = 30	Test Case 2 – Power Enter two numbers to apply Power : 2 5 32	Test Case 3 – Division Enter Dividend and Divisor : 14 3 4.66667

9.8 Time & Space Complexity

Operation	Time Complexity	Space Complexity
Multiplication	O(log x)	O(log x)
Exponentiation	O(log y)	O(log y)
Division	O(log x)	O(log x)

At every recursive call, the input size is reduced by half. Therefore, the recursion depth becomes $\log_2(n)$.

9.9 MCQs

1. Which strategy is used in these programs?	A) Greedy B) Divide and Conquer C) Dynamic Programming D) Backtracking	Answer: B
2. In Power(x,y), when y is even:	A) x is added B) y is doubled C) exponent is halved D) recursion stops	Answer: C
3. Time complexity of Power using D&C is:	A) O(n) B) O(n^2) C) O(log n) D) O(1)	Answer: C
4. Recursive functions require:	A) loops only B) stack memory C) arrays only D) pointers only	Answer: B
5. Divide and Conquer reduces problem size by:	A) doubling B) sorting C) halving D) merging	Answer: C

9.10 Short Questions & Answers

Q1. What is Divide and Conquer?	Divide and Conquer is a technique where a problem is divided into smaller sub-problems, solved recursively, and combined.
Q2. Why is Power using D&C faster than normal multiplication?	Because the exponent is halved at every recursive call, reducing operations significantly.
Q3. What is the time complexity of recursive multiplication?	The time complexity is O(log n) because the value is halved in every recursive call.
Q4. Why is recursion used in Divide and Conquer?	Recursion naturally solves smaller instances of the same problem.
Q5. What is the space complexity of recursive algorithms?	Space complexity depends on recursion depth and is generally O(log n).

9.11 Numericals

1. Find 2^5 using Divide and Conquer.	2^5 = (2×2)^2 × 2 = 4^2 × 2 = 16 × 2 = 32
2. Compute 10 × 3 using recursive multiplication.	10 × 3 = 2 × (5 × 3) = 2 × [2 × (2 × 3) + 3] = 2 × (6 + 3) = 18 Correction after recursive expansion: 10 × 3 = 30
3. Compute 14 / 3 using recursive division.	14/3 = 2 × (7/3) = 2 × [2 × (3/3) + 1/3] = 2 × (2 + 1/3) = 4 + 2/3 = 4.66667

```cpp
/*AIM- To implement multiplication (X*Y), exponentiation (X^Y), and d
approach in C++, and analyze their time and space complexities.*/
//PROGRAMMER NAME = POONIS

#include<iostream>
using namespace std;

int multiply(int x, int y){
    if(x == 0 || y == 0){
        return 0;
    }

    if(x == 1){
        return y;
    }

    int temp = 2 * multiply(x / 2, y);

    if(x % 2 != 0){
        temp = temp + y;
    }

    return temp;
}

long long power(long long x, int y){
    if(y == 0){
        return 1;
    }

    if(y == 1){
        return x;
    }

    if(y % 2 == 0){
        return power(x * x, y / 2);
    }

    return power(x * x, (y - 1) / 2) * x;
}

double divide(double x, double y){
    return x / y;
}

int main(){
    int x, y;

    cout << "Enter Value of X : ";
    cin >> x;

    cout << "Enter Value of Y : ";
    cin >> y;

    cout << "X * Y = " << multiply(x, y) << endl;

    cout << "X ^ Y = " << power(x, y) << endl;

    if(y == 0){
        cout << "Division by Zero Not Allowed" << endl;
    }
    else{
        cout << "X / Y = " << divide(x, y) << endl;
    }

    return 0;
}
```

```
PS C:\out> C:\out\AirthmaticOperation.exe
Enter Value of X : 10
Enter Value of Y : 3
X * Y = 30
PS C:\out> C:\out\AirthmaticOperation.exe
Enter Value of X : 2
Enter Value of Y : 5
X ^ Y = 32
PS C:\out> C:\out\AirthmaticOperation.exe
Enter Value of X : 14
Enter Value of Y : 3
X / Y = 4.66667
```